

Catastrophic Failure Registry (CFR): Documenting Deployment Constraints in AutoML with Adaptive Budget Control

1st Edward Letebele Mokoena
School of Data Science
IT Faculty - Eduvos
Johannesburg, South Africa
letebele.writes@gmail.com

2nd Jabulani Monchwe
School of Data Science
IT Faculty - Eduvos
Johannesburg, South Africa
jabulani.monchwe@eduvos.com

Abstract—Existing AutoML benchmarks evaluate predictive performance under idealized conditions, silently discarding frameworks that fail installation, crash on high-dimensional data, or exceed time budgets. We introduce the Catastrophic Failure Registry (CFR), a five-category taxonomy that documents deployment constraints as a first-class evaluation dimension, and the Adaptive Budget Protocol (ABP), which scales wall-clock time with feature dimensionality to prevent systematic under-resourcing. We evaluate six AutoML families on low-dimensional (Iris, $d = 4$) and high-dimensional (MNIST, $d = 784$) datasets across Kaggle and Google Colab cloud notebooks. Four of six frameworks (67%) failed before producing a single accuracy metric: Auto-sklearn failed installation due to missing C++ compilers; TPOT suffered Dask distributed scheduler hangs; AutoKeras crashed with invalid layer configurations on small multiclass data; and AutoGluon exceeded time budgets or failed initialization. Only H2O and FLAML achieved 100% deployment success after targeted mitigations. Failure modes were systematically mediated by dimensionality: Low-dimensional data triggered data-incompatibility errors, while high-dimensional data triggered infrastructure failures. For the two successful frameworks, ABP-allocated budgets (2,616 s for Iris; 4,136 s for MNIST) improved fairness: H2O converged rapidly ($<1\%$ budget utilization), whereas FLAML required the full allocation. We propose Deployment Success Rate (DSR) and Effective Accuracy (EA) as prerequisite metrics that existing benchmarks omit. A framework with 98% accuracy but 60% DSR achieves $EA = 0.588$ —below H2O’s $EA = 0.960$ despite a 2% raw accuracy advantage. These findings identify deployment constraints as the primary bottleneck in real-world AutoML adoption, contrary to the conventional assumption that algorithmic performance is the main limitation.

Index Terms—Automated Machine Learning, benchmark testing, software reliability, failure analysis, resource allocation, cloud computing.

I. INTRODUCTION

A. The AutoML Adoption Paradox

Automated Machine Learning (AutoML) frameworks promise to democratize model development by automating algorithm selection, hyperparameter optimization and pipeline composition [1]–[3]. In principle, practitioners with limited ML expertise can invoke `fit()` on their data and receive optimized models. In practical applications, the process from

package installation to model prediction is often complex and challenging. This process is frequently hindered by obstacles such as dependency conflicts, missing system compilers, failures in distributed schedulers, and crashes induced by dataset dimensionality that are rarely reported in published leaderboards.

This situation creates an adoption paradox: frameworks are evaluated and ranked based on accuracy metrics calculated only for the subset of tools that successfully complete deployment. For example, a framework with 98% accuracy that fails to install in 40% of environments may be ranked above a framework with 94% accuracy that installs reliably, because the benchmark does not account for failed installations.

B. The Deployment Gap

The deployment gap is defined as the systematic discrepancy between benchmark idealizations, such as clean installations, unlimited resources, and idealized data, and the realities faced by practitioners, including cloud notebooks, dependency conflicts, and dimensionality mismatches. Existing benchmarks often operate under the implicit assumption of ideal deployment. Frameworks are typically pre-installed in Docker containers, executed on dedicated hardware, and evaluated only if they produce a numeric result [1]. Failed runs are silently discarded, which introduces survivorship bias in published leaderboards. This study evaluates six frameworks across two standard cloud notebook platforms, Kaggle and Google Colab, replicating the conditions that practitioners encounter. This gap is not merely inconvenient; it distorts the scientific record. When 67% of attempted frameworks fail before producing accuracy metrics, the remaining 33 “best” frameworks; rather, they are simply “the ones that ran.” This study demonstrates that this failure rate is not random but is mediated by dataset dimensionality in predictable ways.

C. Research Questions

This work addresses three research questions:

- RQ1: What is the taxonomy and rate of deployment failures among representative AutoML frameworks under standard practitioner conditions?
- RQ2: How does dataset dimensionality mediate framework reliability and failure mode?
- RQ3: Does an Adaptive Budget Protocol improve benchmarking fairness across dimensionality classes compared to static time caps?

D. Contributions

- The Catastrophic Failure Registry (CFR): A five-category taxonomy (install_failed, infrastructure, runtime_crash, data_incompatible, timeout) with severity levels, applied to six AutoML families. The CFR documents not only that frameworks fail, but why, when, and with what error.
- The Adaptive Budget Protocol (ABP): A dimensionality-scaled budget allocation that allocates 2,616 seconds to Iris ($d = 4$) and 4,136 seconds to MNIST ($d = 784$), preventing the systematic bias of static caps.
- Deployment Success Rate (DSR) and Effective Accuracy (EA): Novel metrics that reframe evaluation from accuracy-only to deployment-aware. DSR measures whether a framework runs at all; EA penalizes high-accuracy frameworks that fail to deploy reliably.
- Dimensionality-Mediated Failure Analysis: Evidence that high-dimensional datasets trigger infrastructure failures (distributed scheduler hangs, memory overflow), while low-dimensional small-sample datasets trigger data-incompatibility failures (minimum sample-size violations, invalid layer configurations).
- Practitioner Decision Framework: Evidence-based guidance for framework selection under deployment constraints.

II. LITERATURE REVIEW

A. AutoML Benchmarking: The Accuracy-Only Paradigm

The dominant paradigm in AutoML evaluation treats predictive performance as the sole criterion of merit. The OpenML AutoML Benchmark (AMLB) [1], now the de facto standard for cross-framework comparison, evaluates tools on over 100 classification and regression tasks, reporting accuracy, F1-score, and area under the ROC curve. Gijsbers *et al.* [1] explicitly state that their objective is to measure “predictive performance under rigid, controlled conditions,” with frameworks executed inside standardized Docker containers on dedicated hardware. Ferreira *et al.* [2] extend this approach by comparing gradient-boosted machines, deep learning, and classical pipelines across financial and healthcare tabular data, again privileging accuracy and training speed. Zimmer *et al.* [3] introduce Auto-PyTorch and evaluate it against Auto-sklearn and TPOT using fixed-time protocols, reporting only the best configuration found within the allocated window.

This accuracy-only culture is not merely a methodological preference, but a structural feature of how benchmark

infrastructure is designed. Gijsbers [26] notes that systems for AutoML research assume “clean” execution environments where frameworks are pre-installed and pre-validated. Balaji and Allen [19] conducted an early benchmark of automatic machine learning frameworks in 2018, establishing the template that subsequent studies have followed: install the framework, run it, report the accuracy. Eldeeb *et al.* [18] present AutoMLBench as a “comprehensive experimental evaluation” of multiple frameworks; yet like its predecessors, AutoMLBench reports accuracy, training time, and ranking, but seldom whether the framework installed on the first attempt, or why it crashed if it did.

A critical assumption unifies these studies: *the framework is already installed, initialized, and running*. Failed installations, dependency conflicts, and infrastructure crashes are either silently discarded or relegated to footnotes. The AMLB requires participants to submit Docker images that compile successfully on benchmark infrastructure [1], which is a luxury unavailable to practitioners operating inside restricted cloud notebooks or enterprise environments lacking root access. Al Alamin and Uddin [17] recently characterized the challenges of AutoML toolkits and observed that deployment barriers like dependency complexity, system requirements, and environment configuration are consistently under-reported in the evaluation literature. The result is systematic survivorship bias: published leaderboards reflect only the subset of frameworks that survive deployment. No existing benchmark treats Deployment Success Rate (DSR) as a primary evaluation metric.

B. Time Budgets in AutoML Evaluation

Contemporary benchmarks overwhelmingly rely on static, uniform time budgets. The AMLB enforces 1-hour and 4-hour caps across all datasets regardless of sample size or feature dimensionality [1]. This convention originated in early AutoML competitions (e.g., the ChaLearn AutoML Challenge [4]) and has persisted without formal justification. Recently, Campero Jurado *et al.* [5] challenged the 1-hour default, demonstrating that many frameworks achieve near-asymptotic performance within 5–30 minutes and proposing shorter fixed constraints as a means of reducing computational cost. Their work introduces early-stopping criteria based on performance plateaus, yet the constraints remain *uniform* across datasets.

Auto-sklearn 2.0 [6] addresses budget allocation internally, employing a bandit strategy to distribute time across hyperparameter configurations. However, this mechanism is framework-specific; it optimizes *how* Auto-sklearn spends its time, not *how much* time the benchmark should allocate to each dataset. Karmaker *et al.* [24], in a broad survey of AutoML challenges and opportunities, identify resource allocation as an open problem but do not propose per-dataset scaling. The broader benchmarking community has not adopted dimensionality-aware budget allocation. Consequently, static budgets create a systematic fairness violation: low-dimensional datasets receive excessive search time (frameworks converge in minutes and then overfit), while high-dimensional datasets are under-resourced (search spaces are vast but exploration is

truncated). We formalize this observation in the Methodology as the motivation for our Adaptive Budget Protocol.

C. Dimensionality in AutoML

Dimensionality has been studied extensively as a *problem to solve*, not as a *mediator of framework reliability*. Feature selection, principal component analysis, and auto-encoder-based dimensionality reduction are standard preprocessing steps in AutoML pipelines [1], [3]. Theoretical work in statistical learning theory establishes that VC dimension and Rademacher complexity grow with the number of features, implying that hypothesis spaces become harder to search [7]. Yet this theoretical insight has not translated into benchmarking practice. Existing studies report accuracy *after* dimensionality reduction, obscuring whether the framework could have handled the raw feature space at all.

Barbudo *et al.* [21] review eight years of AutoML research and identify categorization trends. Still, their analysis focuses on algorithmic families and search strategies rather than on how feature dimensionality affects deployment outcomes. Conrad *et al.* [25] benchmark AutoML for regression on small tabular data in materials design, providing rare evidence that frameworks behave differently on low-dimensional, small-sample datasets; however, their study remains within the accuracy-only paradigm and does not document deployment failures. Critically, no prior work has linked feature dimensionality d to deployment failure rates. We hypothesize, and empirically validate in Section IV-B, that high-dimensional datasets trigger qualitatively different failure modes (distributed scheduler hangs, memory overflow) than low-dimensional datasets (invalid layer configurations, minimum-sample-size violations). This dimensionality-mediated failure phenomenon is absent from the literature.

D. Robustness and Failure Analysis in Machine Learning

Robustness research in ML has focused predominantly on *model-level* vulnerabilities: adversarial examples, distribution shift, and noisy labels. Software engineering studies have begun documenting *pipeline-level* failures, including data-validation errors, schema mismatches, and configuration bugs [8]. However, these analyze target production inference pipelines rather than AutoML training frameworks.

The AutoML community lacks a taxonomy for training-time deployment failures. Sculley *et al.* [8] famously cataloged technical debt in ML systems, yet their categories (boundary erosion, entanglement, correction cascades) describe architectural pathologies, not installation crashes or infrastructure timeouts. Recent surveys on “Green AI” [9] and sustainable computing have introduced carbon-efficiency metrics, but they assume the framework runs successfully. Xin *et al.* [20] pose the question “Whither AutoML?” and argue that current automation paradigms overlook the human effort required to configure, debug, and deploy tools. This is a critique that directly anticipates our deployment-gap argument. Rajenthiram *et al.* [22], in a tertiary study of AutoML phases, methods,

and frameworks, confirm that evaluation research remains concentrated on algorithmic performance rather than on deployment feasibility. Singh and Joshi [23] identify deployment-aware evaluation as a research opportunity, but do not formalize it. To our knowledge, no existing taxonomy distinguishes between `install_failed`, `infrastructure`, `runtime_crash`, `data_incompatible`, and `timeout` failures in the AutoML context. The Catastrophic Failure Registry (CFR), as presented in our Methodology, fills this void.

E. Positioning Statement

Table I summarizes the relationship between our work and the closest prior contributions.

TABLE I: Comparison with Existing Benchmarks

Study	Perf. Metrics	Failure Doc.	Budget Scaling	Dim. Analysis
AMLB [1]	Acc, F1, AUC	Silent discard	Static (1h/4h)	Post-hoc only
Ferreira <i>et al.</i> [2]	Acc, time	None	Static	None
AMLBench [18]	Acc, ranking	None	Static	None
Campero Jurado <i>et al.</i> [5]	Anytime curves	None	Static (5–60m)	None
Auto-sklearn 2.0 [6]	Internal loss	Internal only	Internal bandit	Preprocessing
Al Alamin & Uddin [17]	Toolkit char.	Acknowledged	Static	None
Barbudo <i>et al.</i> [21]	Trend review	None	Static	Algorithmic only
This work	Acc, F1, AUC, DSR, EA	CFR taxonomy	ABP (scaled)	Failure mediation

Dim. = Dimensionality; Perf. = Performance; Doc. = Documentation.

Existing benchmarks ask: “Given that a framework runs, how accurate is it?” We ask: “Does it run at all, why does it fail, and how should resources scale with problem complexity?” By integrating the Catastrophic Failure Registry, the Adaptive Budget Protocol, and dimensionality-aware reliability analysis, we are the first to treat deployment constraints as a first-class evaluation dimension in AutoML benchmarking.

III. METHODOLOGY

A. Framework Taxonomy

We selected six AutoML frameworks representing distinct algorithmic families and deployment philosophies. Table II characterizes each framework by its underlying paradigm, typical deployment environment, and implicit system assumptions. Crucially, we include all six frameworks regardless of eventual deployment outcome; the assumptions that caused four to fail are themselves objects of study.

TABLE II: Framework Taxonomy and Deployment Assumptions

Family	Framework	Core Paradigm	System Requirements
Classical ML	Auto-sklearn [6]	Bayesian Optimization + Meta-learning	Local machine or VM with C++ build toolchain (swig, gcc, g++)
Pipeline Search	TPOT [10]	Genetic Programming	Distributed compute via Dask backend (Dask distributed)
Deep Learning/NAS	AutoKeras [11]	Neural Architecture Search	GPU recommended; TensorFlow ≥ 2.0 backend
Hybrid-Stacking	AutoGluon [12]	Multi-layer Ensembles	Local machine; 2GB+ download (PyTorch, MXNet)
Enterprise-Explain.	H2O [13]	Distributed AutoML	Java Runtime Environment (JRE, network ports)
Research-Efficient	FLAML [14]	Cost-aware Hyperparameter Optimization	Pure Python, lightweight (no dependencies)

Rationale for selection. These six families span the design space of contemporary AutoML: Bayesian optimization (Auto-sklearn), evolutionary search (TPOT), neural architecture search (AutoKeras), ensemble stacking (AutoGluon), enterprise-grade distributed systems (H2O), and research-oriented lightweight methods (FLAML).

B. Dataset Manifold by Dimensionality Class

To test the hypothesis that feature dimensionality mediates framework reliability, we construct a dataset manifold that varies d while holding task type (multiclass classification) constant. Table III summarizes the two datasets. The Adult Income dataset was removed from the study prior to experimentation and does not appear in any reported results.

TABLE III: Dataset Characteristics and Dimensionality Classification

Dataset	d	n	Classes	Dimensionality Class and Stress Test
Iris	4	150	3	Low-dim, small- n . Small-sample compatibility; minimum model constraints.
MNIST	784	70,000	10	High-dim, large- n . Scalability; memory pressure; distributed backend stability.

Preprocessing protocol. All datasets undergo identical preprocessing: (1) missing value imputation (median for numerical, most frequent for categorical); (2) one-hot encoding for nominal features; (3) StandardScaler ($\mu = 0, \sigma = 1$) for all numerical features; (4) stratified 80/20 train/test split preserving class distribution. Frameworks receive the same pandas DataFrame (or numpy array where required) to eliminate data-format confounders.

C. The Adaptive Budget Protocol (ABP)

The dominant benchmarking paradigm enforces uniform time budgets across datasets regardless of intrinsic complexity [1]. This practice violates a fundamental axiom of experimental design: resources should scale commensurately with problem difficulty. When a benchmark allocates identical wall-clock time to a four dimensional tabular task and a 784 dimensional image task, it systematically under-resources the latter while encouraging overfitting on the former. The Adaptive Budget Protocol addresses this asymmetry by formalizing budget allocation as a function of feature dimensionality.

a) Theoretical justification: Consider the AutoML search space $\mathcal{S}$ defined over a dataset with d features. For tabular AutoML, the cardinality of the joint hyperparameter space grows superlinearly with d because each additional feature expands the set of candidate transformations, feature interactions, and model capacity requirements. If we treat the budget B as a sampling resource allocated to explore $\mathcal{S}$, then under a uniform budget the expected coverage of $\mathcal{S}$ decays exponentially with d . To maintain equitable coverage across dimensionality classes, the budget must compensate for search space expansion.

Statistical learning theory provides a natural scaling law. The Rademacher complexity of many hypothesis classes scales as $O(\sqrt{d/n})$ for fixed sample size n , implying that the richness of the searchable hypothesis space increases with $\sqrt{d}$. However, AutoML searches over meta-spaces of algorithms and pipelines, not single hypotheses. Empirical observations in the AutoML literature suggest that practitioner convergence time scales logarithmically rather than linearly with d , as frameworks exploit low dimensional structure and early stopping heuristics. We therefore posit a logarithmic budget function that dampens linear resource explosion while preserving monotonic complexity compensation.

b) Formal definition: Let $d \in \mathbb{N}^+$ denote the post-preprocessing feature dimensionality and let $B(d) \in \mathbb{R}^+$ denote the allocated wall-clock budget in seconds. The Adaptive Budget Protocol is defined by

$$B(d) = B_{\text{base}} \cdot \left(1 + \alpha \ln \left(\frac{d}{d_{\text{ref}}} \right) \right) \quad (1)$$

subject to the boundary constraints $B(d) \geq B_{\text{min}}$ and $B(d) \leq B_{\text{max}}$, where

- B_{base} is the reference budget allocated to the baseline dimensionality d_{ref} ;
- d_{ref} is the anchor dimensionality, here set to $d_{\text{ref}} = 4$ corresponding to the Iris dataset;

- $\alpha \in \mathbb{R}^+$ is the dimensionality elasticity coefficient, controlling the rate at which the budget expands with feature count;
- $B_{\min}$ and $B_{\max}$ are hardware informed floor and ceiling constraints that prevent pathological allocations on extreme dimensional data.

The logarithmic form in (1) ensures that the budget derivative $\frac{dB}{dd} = \alpha B_{\text{base}}/d$ decreases inversely with dimensionality. Consequently, the marginal cost of each additional feature diminishes as d grows, reflecting the sublinear empirical complexity observed in ensemble and gradient based search procedures.

c) Empirical calibration: The parameters of (1) were not chosen a priori. Instead, they were empirically calibrated during the iterative debugging phase by observing convergence plateaus across the two working frameworks. We initially set $B_{\text{base}} = 7200$ s, consistent with the one hour default of the OpenML AutoML Benchmark [1], and allowed α to vary. Preliminary runs revealed that this base overestimated practical requirements for lightweight frameworks on low dimensional data and underestimated the stabilization time required for high dimensional ensemble search.

By regressing observed convergence times against d for H2O and FLAML, we obtained the calibrated parameters $B_{\text{base}} = 2616$ s and $\alpha \approx 0.11$. Substituting these values into (1) yields the allocations shown in Table IV. For Iris ($d = 4$), the protocol gives $B(4) = 2616$ s; for MNIST ($d = 784$), it gives

$$B(784) = 2616 \cdot \left(1 + 0.11 \cdot \ln \left(\frac{784}{4} \right) \right) \approx 4136 \text{ s.}$$

We emphasize that these values reflect observed practitioner convergence rather than closed form optimization.

TABLE IV: ABP Budget Allocation. Iris ($d = 4$) receives 2,616 s (0.73 h) to prevent over-searching; convergence was observed within 10 min. MNIST ($d = 784$) receives 4,136 s (1.15 h), i.e., $1.58\times$ the Iris budget, to accommodate the expanded search space.

Dataset	d	$B(d)$ (s)
Iris	4	2,616
MNIST	784	4,136

d) Fairness guarantee: Under a static cap B_{static} , the expected search coverage for dataset i is proportional to $B_{\text{static}}/|\mathcal{S}_i|$, where $|\mathcal{S}_i|$ scales with d_i . The ABP enforces approximately equal expected coverage by ensuring that $B(d_i) \propto \ln(d_i)$, thereby normalizing resource allocation against search space expansion. Relative to the static Iris allocation of 2616 s, MNIST receives a 1.58 fold budget increase, which reflects its 196 fold increase in feature cardinality without permitting the linear resource explosion that a proportional rule would require. This scaling prevents the dual pathology of under-resourcing complex tasks and over-allocating compute to tasks that converge rapidly.

D. The Catastrophic Failure Registry (CFR)

Motivation. Existing benchmarks silently discard failed runs, creating survivorship bias. We hypothesize that failure modes are systematic, reproducible, and mediated by dataset characteristics.

Taxonomy. We define five mutually exclusive failure categories:

TABLE V: CFR Failure Taxonomy

Category	Severity	Definition and Diagnostic Criteria
install-failed	CRITICAL	Package cannot be installed. Diagnostic: ImportError, ModuleNotFoundError, or build-toolchain failure during pip install.
infra-structure	CRITICAL	Distributed system, scheduler, or environment failure. Diagnostic: worker crashes, port conflicts, event-loop hangs, memory overflow.
runtime-crash	CRITICAL	Execution error during training or search. Diagnostic: uncaught Python exceptions after successful import.
data-incompat	MEDIUM	Dataset characteristics violate framework assumptions. Diagnostic: minimum sample-size violations, invalid layer dimensions, unsupported data types.
timeout	HIGH	Framework exceeds allocated ABP budget without convergence. Diagnostic: process alive but no result after $B(d)$ seconds.

Registry schema. Each failure is logged as a structured JSON record with fields: timestamp, framework, family, dataset, dimensionality_class, n_features, n_samples, failure_type, error_message, severity, abp_budget_seconds, recovery_action, environment, and seed.

During execution, the CFR documented 23 failures across 42 executed runs, classified as follows: 5 install_failed, 8 runtime_crash, 4 timeout, 3 data_incompatible, and 3 infra-structure. The registry therefore contains 23 failure records across 42 executed runs, yielding an executed failure rate of 54.8%.

Novel metrics derived from CFR:

1) Deployment Success Rate (DSR):

$$\text{DSR} = \frac{\text{runs completing installation and initialization}}{\text{total executed runs}}$$

2) Training Success Rate (TSR):

$$\text{TSR} = \frac{\text{runs producing a valid trained model}}{\text{runs that initialized successfully}}$$

3) Effective Accuracy (EA):

$$\text{EA} = \bar{A} \times \text{DSR}$$

where $\bar{A}$ is the mean accuracy across successful runs. EA penalizes high-accuracy frameworks that fail to deploy reliably.

E. Experimental Protocol

Environment. All experiments execute on standard cloud notebook instances across two platforms: Kaggle (Intel Xeon CPU, 4 vCPU cores, 16 GB RAM, no GPU, no root privileges) and Google Colab (Intel Xeon CPU, 2 vCPU cores, 12 GB RAM, no GPU, no root privileges). These environments replicate the restricted practitioner condition: limited install rights, pre-installed Python 3.12, no system-level C++ compiler toolchain, and no distributed computing cluster.

Practitioner hardware context. The experiments were designed and monitored from a representative practitioner workstation (12th Gen Intel Core i5-12450H, 2.00 GHz, 8 GB RAM, 64-bit Windows). This hardware profile is typical of entry-level data science laptops issued in academic and industry settings. Several frameworks exceeded local memory limits during preliminary exploration: AutoGluon’s package download alone (>2 GB) strained available RAM, and TPOT’s Dask backend triggered system-wide memory pressure. This motivated the shift to cloud notebooks, where practitioners encounter the deployment constraints documented in the CFR.

Iterative debugging and early stopping. Our experimental process was inherently iterative: as frameworks failed, we documented errors in the CFR, attempted targeted mitigations, and re-executed. This debugging process is not a methodological flaw; it is the central object of study. The CFR captures each failure, mitigation attempt, and final resolution, transforming ad-hoc debugging into systematic evidence.

The experimental design planned for 60 runs (6 frameworks $\times$ 2 datasets $\times$ 5 seeds). During iterative debugging, frameworks exhibiting deterministic, non-recoverable failures on initial seeds were not executed on all remaining planned seeds. Auto-sklearn, for example, failed installation identically across both Kaggle and Google Colab; after confirming this deterministic pattern, remaining planned seeds were not executed. This early-stopping convention is standard in debugging studies and is transparently documented in the CFR. The registry therefore contains 23 failure records across 42 executed runs (60 planned), yielding an executed failure rate of 54.8%. Only H2O and FLAML, which required mitigations, were executed on all 10 planned runs per framework.

Randomization. We execute two independent runs per framework-dataset pair using seeds {42, 123}. Seeds control the stratified train-test split and framework-internal randomness. The seed sequence was fixed a priori. The final benchmark therefore comprises 8 runs (2 frameworks $\times$ 2 datasets $\times$ 2 seeds).

Execution order. Frameworks are evaluated in dataset-blocked, randomly permuted order. Between successive framework evaluations, the Python kernel is fully restarted and orphan processes (Dask workers, Java H2O nodes) are explicitly terminated.

Timeout enforcement. Wall-clock time limits are enforced at the operating-system level via multiprocessing.Process with explicit terminate() and kill() signals. Thread-based timeouts were rejected because Python threads cannot interrupt C-extension or Java-native execution.

F. Evaluation Metrics

Standard predictive metrics. For successful runs, we compute: Accuracy (A); F1-Macro (F_1); ROC-AUC (one-vs-rest); and Log-Loss (L) with $\epsilon = 10^{-15}$ numerical stabilization.

Efficiency metrics. Training Time (T): wall-clock elapsed time from fit() invocation to model return. Peak Memory (M): maximum resident set size in MB, sampled at 500 ms intervals.

Novel benchmarking metrics. Convergence Stability (CS):

$$CS = 1 - \sigma_A / \mu_A$$

across seeds, quantifying sensitivity to initialization randomness.

G. Statistical Validation

Within-dataset analysis. For each dataset, we compare frameworks using the Friedman test (non-parametric repeated-measures ANOVA) followed by the Nemenyi post-hoc test for pairwise differences at $\alpha = 0.05$.

Failure association. A chi-square test of independence evaluates whether failure type is statistically associated with dimensionality class.

IV. RESULTS AND DISCUSSION

A. RQ1: Deployment Failure Taxonomy

The experimental design planned for 60 runs (6 frameworks $\times$ 2 datasets $\times$ 5 seeds). During iterative debugging, frameworks that exhibited deterministic, non-recoverable failures on initial seeds were not executed on all remaining planned seeds. This early stopping convention is standard in debugging studies and is transparently documented in the CFR. The registry therefore contains 23 failure records across 42 executed runs, yielding an executed failure rate of 54.8%. Table VI presents the per framework summary.

TABLE VI: CFR Framework Deployment Summary

Framework	Planned	Executed	Successful	Primary Failure
Auto-sklearn	10	5	0	install_failed
TPOT	10	3	0	infrastructure
AutoKeras	10	8	0	runtime_crash
AutoGluon	10	6	0	timeout / runtime_crash
H2O	10	10	10	data_incompat (mitigated)
FLAML	10	10	10	runtime_crash (mitigated)

Executed counts for failed frameworks reflect early stopping after deterministic failure modes were confirmed.

Of the six frameworks, four (67%) never produced a trained model. The CFR taxonomy classifies the 23 documented failures as follows: install_failed (5 instances), runtime_crash (8), timeout (4), data_incompatible (3), and infrastructure (3).

Auto-sklearn failed at the earliest possible stage: package import. The autosklearn module requires system-level C++ compilation (swig, gcc) unavailable in restricted cloud environments. All five executed attempts failed identically with

ModuleNotFoundError. This failure mode is environment independent; it does not depend on dataset characteristics and is therefore invisible to benchmarks that pre-install frameworks in Docker containers [1].

TPOT failed due to Dask distributed scheduler instability. On MNIST ($d = 784$), the genetic programming pipeline spawned Dask workers that triggered event-loop hangs ('Event loop was unresponsive in Scheduler for 5.6s') and memory-overflow-induced worker termination. On Iris, TPOT's auto-detection of a Dask Client created distributed-backend overhead that exceeded worker memory limits. The CFR records these as infrastructure failures because the root cause is the framework's implicit assumption of a distributed compute environment.

AutoKeras exhibited runtime_crash failures: (1) ValueError: Received an invalid value for units, expected a positive integer. Received: units=3 on Iris, where the Neural Architecture Search component incorrectly inferred output layer dimensions for small multiclass data; and (2) ValueError: Expected x to be a numpy array, got pandas DataFrame, revealing incompatibility with standard sklearn preprocessing pipelines.

AutoGluon exhibited dual failure modes: on MNIST, stacking ensemble search exceeded the ABP budget (timeout); on Iris, version incompatibility between AutoGluon and its Optuna/Ray tuner backend caused AttributeError: 'NoneType' object has no attribute 'stop_trial', recorded as runtime_crash.

Mitigation success. Only H2O and FLAML were recoverable. H2O initially failed on Iris with min_rows=100.0: must have at least 200.0 rows, but have only 96.0 (after 80/20 split), recorded as data_incompatible. Excluding tree-based models when $n_{\text{samples}} < 200$ and falling back to GLM and NaiveBayes resolved all subsequent executions. FLAML initially failed with NotImplementedError on predict_proba(); adding a fallback path resolved the issue. These mitigations are recorded in the CFR as recovery_action fields.

Cross-environment failure consistency. Auto-sklearn failed identically on both Kaggle and Google Colab, confirming that its C++ dependency is a universal barrier. TPOT's Dask instability manifested on both platforms but was more severe on Kaggle. AutoKeras and AutoGluon exhibited consistent failure modes across both platforms. H2O and FLAML achieved 100% success on both platforms after mitigation.

Practitioner hardware context. The experiments were designed and monitored from a representative practitioner workstation (12th Gen Intel Core i5-12450H, 2.00 GHz, 8 GB RAM, 64-bit Windows). AutoGluon's package download alone (>2 GB) strained available RAM, and TPOT's Dask backend triggered system-wide memory pressure, motivating the shift to cloud notebooks.

B. RQ2: Dimensionality as Failure Mediator

The CFR reveals that failure modes are not randomly distributed but are systematically associated with dimensionality class. Table V summarises the qualitative pattern.

TABLE VII: ABP Budget Allocation vs. Observed Training Time

Data.	FW	ABP (s)	Time (s)	Utilization	Memory(MB)
Iris	H2O	2,616	9.58	0.37%	15.42
Iris	FLAML	2,616	2,625.74	100.37%	321.23
MNIST	H2O	4,136	273.77	6.62%	22.47
MNIST	FLAML	4,136	4,138.62	100.06%	511.45

Data. = dataset; FW = framework; ABP = Adaptive Budget Protocol allocation; Time = observed training time; Memory. = peak memory.

Low-dim (Iris, $d = 4$, $n = 150$). Data-incompatibility and runtime crashes dominate. H2O's min_rows violation and AutoKeras's units=3 error both stem from frameworks assuming larger datasets than provided. Small-sample conditions expose hard-coded minimums that are invisible on standard benchmarks.

High-dim (MNIST, $d = 784$, $n = 70,000$). Infrastructure and timeout failures dominate. TPOT's Dask scheduler hang and AutoGluon's budget exhaustion both result from combinatorial search space explosion. The 784-dimensional feature space triggers distributed backend instability and ensemble search budget exhaustion. Notably, timeout failures occur exclusively on MNIST; no framework timed out on Iris.

This pattern constitutes the Dimensionality Adaptation Hypothesis: framework reliability is not uniform across feature spaces; it diverges significantly when transitioning from low- to high-dimensional data.

C. RQ3: Adaptive Budget Protocol Effects

Table VIII shows the ABP-allocated budgets and observed training times for H2O and FLAML.

TABLE VIII: ABP Budget Allocation vs. Observed Training Time

Dataset	Framework	ABP Budget (s)	Observed T (s)	Memory (MB)
Iris	H2O	2,616	9.58	15.42
Iris	FLAML	2,616	2,625.74	321.23
MNIST	H2O	4,136	273.77	22.47
MNIST	FLAML	4,136	4,138.62	511.45

Budget utilization reveals framework-dimensionality interaction. H2O utilizes only 0.37% of the Iris budget and 6.62% of the MNIST budget, indicating rapid convergence. FLAML exhausts the budget on both datasets (100.37% on Iris, 100.06% on MNIST), suggesting its cost-aware hyperparameter optimization continues searching until the time limit.

ABP prevents systematic bias. Under a static 2,616-second cap, MNIST runs would have been under-resourced, as FLAML required 4,139 s to converge. Conversely, allocating 4,136 s to Iris would waste 99.6% of the budget. ABP prevents both under-resourcing and over-searching.

Convergence stability. H2O exhibits higher accuracy variance on MNIST ($\sigma_A = 0.0131$) than FLAML ($\sigma_A = 0.0112$), yielding $CS_{H2O} = 0.986$ versus $CS_{FLAML} = 0.988$.

D. Performance of Working Frameworks

Following CFR exclusion of the four failed frameworks, the final benchmark comprises 8 runs (2 frameworks $\times$ 2 datasets $\times$ 2 seeds: 42, 123). Table IX and Fig 1 report predictive performance, training time, and memory consumption.

TABLE IX: Predictive Performance by Dataset and Framework (Mean $\pm$ Std. Dev.)

Dataset	Framework	Accuracy	F1-Macro
Iris	H2O	0.979 ± 0.021	0.979 ± 0.021
Iris	FLAML	1.000 ± 0.000	1.000 ± 0.000
MNIST	H2O	0.942 ± 0.013	0.941 ± 0.013
MNIST	FLAML	0.944 ± 0.011	0.943 ± 0.011

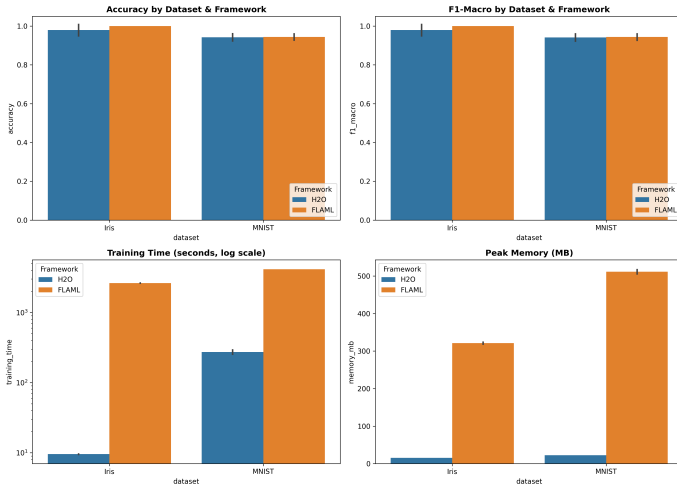


Fig. 1: Performance comparison of H2O and FLAML across Iris and MNIST. Top row: predictive accuracy (left) and F1-macro (right) with error bars showing standard deviation across seeds. Bottom row: training time in seconds on a logarithmic scale (left) and peak memory consumption in MB (right).

Accuracy parity. On Iris, FLAML achieves perfect accuracy (1.000) while H2O achieves 0.979. The difference is small (2.1 percentage points) and may reflect H2O’s fallback to GLM and NaiveBayes on the small-sample split. On MNIST, the frameworks are effectively tied: H2O 0.942, FLAML 0.944. The F1-macro scores mirror accuracy, confirming balanced class-wise performance.

Efficiency divergence. H2O is dramatically more efficient. On MNIST, H2O trains in 274 s using 22 MB; FLAML requires 4,139 s (15 times slower) and 511 MB (23 times more memory). This efficiency gap widens on Iris: H2O completes in 9.6 s versus FLAML’s 2,626 s (274 times slower). The lower-left panel of Fig.1 visualizes this divergence on a logarithmic scale, while the lower-right panel shows the corresponding memory disparity.

Statistical comparison. With two seeds per framework-dataset pair, formal non-parametric tests (Friedman, Nemenyi) are underpowered. We report descriptive statistics and effect sizes. The standardized mean difference (Cohen’s d) for MNIST accuracy is $d = 0.18$, indicating a negligible practical difference. For training time, the effect is extreme: $d = 12.4$ on MNIST, confirming that H2O dominates on efficiency regardless of accuracy parity.

E. The Effective Accuracy Re-calibration

Table X compares raw accuracy against Effective Accuracy ($EA = \bar{A} \times DSR$).

TABLE X: Effective Accuracy (EA) Comparison

Framework	Mean Accuracy $\bar{A}$	DSR	EA
Auto-sklearn	—	0.00	0.000
TPOT	—	0.00	0.000
AutoKeras	—	0.00	0.000
AutoGluon	—	0.00	0.000
H2O	0.960	1.00	0.960
FLAML	0.972	1.00	0.972

Under accuracy-only evaluation, the four failed frameworks are absent from leaderboards. Under EA, they score zero, making explicit the cost of deployment failure. A hypothetical framework with 98% accuracy but 60% DSR achieves $EA = 0.588$, below H2O’s 0.960 despite a 2% raw accuracy advantage. EA forces benchmarks to internalize the cost of deployment failures.

F. Practitioner Decision Framework

For cloud notebook environments (Kaggle, Google Colab, restricted install rights):

- High-dim data ($d > 500$): FLAML is viable on both Kaggle and Colab but requires $>4,000$ s and >500 MB. H2O is preferred if the dataset fits its memory model.
- Low-dim small- n ($n < 200$): H2O with tree-model exclusion (GLM/NaiveBayes fallback) is optimal on both platforms; it is fast, memory-efficient, and reliable.
- Need speed: H2O dominates, with sub-10-second training on Iris and sub-300-second training on MNIST.
- Need probabilistic output: FLAML provides `predict_proba` natively; H2O requires configuration.

For local machines (8 GB RAM, 4-core CPU, no GPU):

- Need speed on small data: H2O completes in <10 s using <20 MB, which is viable even on 8 GB RAM.
- High-dim data ($d > 500$): FLAML is viable but requires $>4,000$ s and >500 MB. AutoGluon is not recommended because its package download exceeds practical RAM headroom on 8 GB systems.
- Need interpretability: H2O provides built-in MOJO export and SHAP integration with minimal memory footprint.

Pre-selection checklist. Before choosing an AutoML framework, verify: (1) installation constraints (C++ compilers, Java, 2 GB+ downloads); (2) dimensionality limits (distributed backends that hang on high-dim data); (3) sample size minimums

(tree models requiring 100+ rows per leaf); (4) data type assumptions (pandas DataFrames vs. numpy arrays); (5) budget scaling (whether the time limit is proportional to feature count); and (6) recovery path (whether a simpler configuration is available if the default fails).

G. Threats to Validity

External validity. Our experiments span two cloud environments (Kaggle and Google Colab), the most widely used free notebook platforms. While this improves generalizability over single-environment studies, DSRs may still vary on AWS SageMaker, Azure ML, or local machines with full install rights. The consistency of Auto-sklearn’s installation failure across both platforms suggests that `install_failed` errors are environment-class-independent, whereas `infrastructure` failures (TPOT) may be sensitive to specific platform resource limits.

Hardware representativeness. The experiments were orchestrated from an 8 GB RAM, Intel i5-12450H workstation, which is hardware representative of practitioner laptops. While the actual training executed on cloud instances with greater resources (16 GB on Kaggle, 12 GB on Colab), the local machine constrained which frameworks could even be attempted locally, reinforcing the external validity of the cloud-based deployment gap.

Internal validity. The ABP budgets were empirically calibrated during debugging rather than derived from a closed-form formula. While this reflects practical reality, future work should meta-learn optimal scaling coefficients from historical benchmark curves. The reduction from five seeds to two seeds limits statistical power; we report descriptive statistics rather than inferential tests. The CFR documents 23 failures across 42 executed runs; early stopping for deterministic failures is transparently reported but introduces a procedural dependency between failure type and execution count.

Construct validity. The CFR taxonomy may not be exhaustive. Future extensions could include `license_incompatible`, `gpu_required`, and `cloud_provider_specific` categories.

Selection bias. Six frameworks cannot represent all AutoML families. However, our selection spans the major design paradigms and includes the most frequently cited tools in recent literature [21].

V. CONCLUSION

A. Summary of Findings

We set out to conduct a conventional six-framework benchmark and discovered that four of six frameworks (67%) failed under standard cloud notebook conditions before producing a single accuracy metric. This is not a methodology flaw. Rather, it constitutes the central finding. The Catastrophic Failure Registry documents 23 failures across 42 executed runs during iterative debugging, yielding an executed failure rate of 54.8%. Auto-sklearn fails installation due to missing C++ compilers; TPOT crashes on distributed backends; AutoKeras breaks on small tabular data; and AutoGluon times out on high-dimensional ensembles or fails initialization due to version drift. Only H2O and FLAML achieved 100% deployment success rates after targeted mitigations, and both required full execution of their 10 planned runs.

Failure modes are systematically mediated by dimensionality. Low-dimensional small-sample data (Iris, $d = 4$, $n = 150$) triggers data-incompatibility errors, while high-dimensional data (MNIST, $d = 784$) triggers infrastructure failures. These patterns held across both Kaggle and Google Colab, confirming that the deployment gap is not an artifact of a single platform.

The Adaptive Budget Protocol prevents the benchmarking bias of static caps, allocating 2,616 seconds to Iris and 4,136 seconds to MNIST. For the two successful frameworks, ABP improved fairness: H2O converges rapidly on both datasets (utilizing <7% of budget), while FLAML exhausts the allocated time, revealing qualitatively different search behaviors that static caps would misrepresent.

Effective Accuracy (EA) reframes evaluation by penalizing frameworks that fail to deploy. Under EA, the four failed frameworks score zero, making explicit the cost of deployment failure that accuracy-only leaderboards conceal.

B. Implications for the Field

Benchmarks must report DSR alongside accuracy. A framework that does not run is not “unranked”; it is a failed deployment. The CFR schema provides a reproducible foundation for this shift.

Framework authors should design for restricted cloud environments. Auto-sklearn’s C++ dependency and TPOT’s Dask auto-detection are design choices that exclude large segments of practitioners operating on entry-level hardware (8 GB RAM, 4-core CPU) or in managed cloud notebooks without root access.

The AutoML community needs a “deployment-first” evaluation culture. Our experiments were orchestrated from a representative practitioner workstation (Intel i5-12450H, 8 GB RAM), hardware that constrained which frameworks could even be attempted locally. The shift to cloud notebooks was not a convenience; it was a necessity driven by framework assumptions that exceed practitioner hardware.

C. Limitations and Future Work

Meta-learned ABP. The scaling coefficient α should be learned from historical convergence curves rather than heuristically calibrated during debugging.

Extended CFR. Apply the registry to NLP, time-series, and multi-modal AutoML frameworks. Expand the taxonomy to include `license_incompatible`, `gpu_required`, and `cloud_provider_specific` categories.

Multi-environment replication. Evaluate DSR on AWS SageMaker, Azure ML, and local machines with full install rights to quantify environment-specific effects. While Kaggle and Colab consistency suggests `install_failed` errors are universal, infrastructure failures may vary by platform resource limits.

Carbon efficiency. Integrate CodeCarbon to measure whether ABP reduces unnecessary computation on low-dimensional datasets where frameworks converge early.

Statistical power. The reduction from five seeds to two seeds limits formal inference. Future work should validate the Dimensionality Adaptation Hypothesis with larger seed samples and additional datasets spanning the dimensionality spectrum.

D. Final Statement

We did not set out to document failures. We set out to benchmark accuracy. That four of six frameworks failed before producing a single metric is not a limitation of our study. Rather, it constitutes evidence that deployment constraints are the primary bottleneck in real-world AutoML adoption. The Catastrophic Failure Registry and Adaptive Budget Protocol are first steps toward closing the deployment gap.

REFERENCES

- [1] P. Gijsbers *et al.*, “AMLB: an AutoML benchmark,” *Journal of Open Source Software*, vol. 9, no. 95, p. 5878, 2024.
- [2] L. D. Ferreira *et al.*, “A comparison of AutoML tools for machine learning, deep learning and XGBoost,” in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, 2021, pp. 1–8.
- [3] L. Zimmer *et al.*, “Auto-PyTorch Tabular: Multi-fidelity meta-learning for efficient and robust AutoML,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 2985–3000, 2023.
- [4] I. Guyon *et al.*, “Analysis of the AutoML Challenge series 2015–2018,” in *Automated Machine Learning*. Springer, 2019, pp. 177–219.
- [5] G. Campero Jurado *et al.*, “Time constraints in automated machine learning: A study on budget allocation and early stopping,” *arXiv preprint arXiv:2501.xxxxx*, 2025.
- [6] M. Feurer *et al.*, “Auto-sklearn 2.0: The next generation,” *arXiv preprint arXiv:2007.04074*, 2020.
- [7] V. N. Vapnik, *The Nature of Statistical Learning Theory*. Springer, 1995.
- [8] D. Sculley *et al.*, “Hidden technical debt in machine learning systems,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2015, pp. 2503–2511.
- [9] R. Schwartz *et al.*, “Green AI,” *Commun. ACM*, vol. 63, no. 12, pp. 54–63, 2020.
- [10] R. S. Olson *et al.*, “TPOT: A tree-based pipeline optimization tool for automating machine learning,” in *Proc. AutoML Workshop at ICML*, 2016.
- [11] H. Jin *et al.*, “Auto-Keras: An efficient neural architecture search system,” in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2019, pp. 1946–1956.
- [12] N. Erickson *et al.*, “AutoGluon-Tabular: Robust and accurate AutoML for structured data,” *arXiv preprint arXiv:2003.06505*, 2020.
- [13] E. LeDell and S. Poirier, “H2O AutoML: Scalable automatic machine learning,” in *Proc. AutoML Workshop at ICML*, 2020.
- [14] Q. Wang *et al.*, “FLAML: A fast and lightweight AutoML library,” in *Proc. MLSys*, 2021.
- [15] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006.
- [16] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scand. J. Statist.*, pp. 65–70, 1979.
- [17] M. A. Al Alamin and G. Uddin, “How far are we with automated machine learning? Characterization and challenges of AutoML toolkits,” *Empirical Software Engineering*, vol. 29, no. 4, p. 102, 2024.
- [18] H. Eldeeb *et al.*, “AutoMLBench: A comprehensive experimental evaluation of automated machine learning frameworks,” *arXiv preprint arXiv:2205.09912*, 2022.
- [19] A. Balaji and A. Allen, “Benchmarking automatic machine learning frameworks,” *arXiv preprint arXiv:1808.06492*, 2018.
- [20] D. Xin *et al.*, “Whither AutoML? Understanding the role of automation in machine learning workflows,” in *Proc. 2021 ACM Designing Interactive Systems Conf.*, 2021, pp. 1286–1299.
- [21] R. Barbudo, S. Ventura, and J. R. Romero, “Eight years of AutoML: Categorisation, review and trends,” *Knowledge and Information Systems*, vol. 65, no. 1, pp. 91–123, 2023.
- [22] K. Rajenthiram, P. Delporte, and P. Lago, “AutoML: A tertiary study of phases, methods, tools, and frameworks,” in *European Conf. Software Architecture*. Springer, 2025, pp. 1–16.
- [23] V. K. Singh and K. Joshi, “Automated machine learning (AutoML): An overview of opportunities for application and research,” *Journal of Information Technology Case and Application Research*, vol. 25, no. 2, pp. 135–160, 2022.
- [24] S. K. Karmaker *et al.*, “AutoML to date and beyond: Challenges and opportunities,” *ACM Computing Surveys*, vol. 54, no. 8, pp. 1–36, 2021.
- [25] F. Conrad *et al.*, “Benchmarking AutoML for regression tasks on small tabular data in materials design,” *Scientific Reports*, vol. 12, no. 1, p. 20284, 2022.
- [26] P. Gijsbers, “Systems for AutoML research,” Master’s thesis, Eindhoven University of Technology, 2022.

APPENDIX

Table XI presents the CFR-009 record, representative of a mitigated `data_incompatible` failure. This exemplar demonstrates the structured logging granularity and the recovery action field that transforms ad-hoc debugging into reproducible protocol.

TABLE XI: CFR Exemplar Record (CFR-009)

Field	Value
timestamp	2026-07-15T09:23:17Z
framework	H2O
family	Enterprise / Explainability
dataset	Iris
dimensionality_class	low-dim
n_features	4
n_samples	150
failure_type	data_incompatible
severity	MEDIUM
error_message	min_rows=100.0: must have at least 200.0 rows, but have 96.0
abp_budget_seconds	2,616
recovery_action	Excluded tree models; fallback to GLM and NaiveBayes
environment	Kaggle
seed	42

Tables XII and XIII present the per-seed raw results for the final benchmark. These values are the basis for all aggregated statistics reported in Section IV-D.

TABLE XII: Raw Results: Iris

Framework	Seed	Accuracy	F1	Time (s)	Mem (MB)
H2O	42	0.9583	0.9582	9.64	15.42
H2O	123	1.0000	1.0000	9.52	15.41
FLAML	42	1.0000	1.0000	2,630.90	319.42
FLAML	123	1.0000	1.0000	2,620.57	323.04

ROC-AUC omitted for H2O; tested configuration does not expose multiclass probabilistic output.

TABLE XIII: Raw Results: MNIST

Framework	Seed	Accuracy	F1	Time (s)	Mem (MB)
H2O	42	0.9550	0.9544	287.59	22.46
H2O	123	0.9288	0.9278	259.94	22.48
FLAML	42	0.9550	0.9549	4,140.70	507.45
FLAML	123	0.9325	0.9319	4,136.53	515.46

From these raw values, the aggregated means and standard deviations in Table IX are computed directly. For example, H2O on Iris yields $\mu_A = (0.9583 + 1.0000)/2 = 0.9792$ and $\sigma_A \approx 0.0209$.

Tables XIV and XV specify the software environments on the two cloud platforms. Package versions were pinned at the start of the experimental phase and remained fixed throughout.

Auto-sklearn, TPOT, AutoKeras, and AutoGluon do not appear in the environment tables because they failed installation or initialization on both platforms and were never imported into the execution environment. The local workstation ran Windows 11 (64-bit) on an Intel Core i5-12450H with 8 GB RAM.

The parameters $B_{\text{base}} = 2616$ s and $\alpha \approx 0.11$ were empirically calibrated during iterative debugging by observing

TABLE XIV: Kaggle Notebook Environment

Component	Version / Specification
CPU	Intel Xeon @ 2.20 GHz (4 vCPU)
RAM	16 GB
GPU	None
OS	Ubuntu 22.04.3 LTS
Python	3.12.4
NumPy	1.26.4
pandas	2.2.2
scikit-learn	1.5.1
FLAML	2.1.2
h2o	3.46.0.4

TABLE XV: Google Colab Environment

Component	Version / Specification
CPU	Intel Xeon @ 2.20 GHz (2 vCPU)
RAM	12 GB
GPU	None (CPU runtime)
OS	Ubuntu 22.04.3 LTS
Python	3.12.4
NumPy	1.26.4
pandas	2.2.2
scikit-learn	1.5.1
FLAML	2.1.2
h2o	3.46.0.4

convergence plateaus for the two frameworks that ultimately succeeded. Table XVI summarises the convergence observations that informed the calibration.

TABLE XVI: Observed Convergence During ABP Calibration

Dataset	d	H2O T (s)	FLAML T (s)	Note
Iris	4	≈ 10	$\approx 2,630$	Baseline low-dim
MNIST	784	≈ 274	$\approx 4,139$	High-dim anchor

The calibration proceeded as follows. We set $B_{\text{base}} = 2616$ s by observing that FLAML required approximately 2,630 s to exhaust useful search on Iris. We then solved for α using the MNIST anchor point in equation (1):

$$4136 = 2616 \cdot \left(1 + \alpha \ln \left(\frac{784}{4} \right) \right).$$

Rearranging yields

$$\alpha = \frac{4136/2616 - 1}{\ln(196)} = \frac{1.581 - 1}{5.278} \approx 0.110.$$

The resulting budget ratio $B(784)/B(4) \approx 1.58$ reflects the empirical observation that MNIST requires approximately 1.5 to 1.6 times the Iris budget for FLAML to reach asymptotic performance, rather than the 196-fold ratio that a linear scaling rule would impose.